

StockVision Software Testing Chat History

Session ID: session_snake_1784215078178_ed1d46dfd6d40f49

Created: 2026-07-16T15:17:58.177892900Z

Updated: 2026-07-17T03:58:05.233908800Z

Source: C:\Users\user\.jcode\sessions\session_snake_1784215078178_ed1d46dfd6d40f49.json

=====

[2026-07-16T15:19:23.335217900Z] USER

I'm working on a Software Testing (ITS64704) group assignment where I need to create a Software Test Plan and Test Execution Report based on this project. Please give me a comprehensive knowledge dump of the project so I can use it to build these testing documents. Structure your answer with these exact headings:

=====

[2026-07-16T15:19:38.537628500Z] USER

Project Name & One-Line Description

Project Objectives (business goals, what problem it solves)

Target Users / Stakeholders (roles: admin, end user, etc.)

Functional Requirements ? list every feature/module the system has, grouped by module (e.g. Authentication, Dashboard, Data Input, Reporting, etc.), each as a short bullet

Non-Functional Requirements ? performance, security, usability, scalability, availability targets if any

Business Rules / Validation Rules ? e.g. field constraints, workflow rules, access control rules

System Architecture Summary ? frontend stack, backend stack, database, APIs, third-party integrations

User Roles & Permissions ? what each role can/cannot do

Key User Workflows / Use Cases ? step-by-step for the top 5-8 core flows (e.g. login, submit form, generate report, admin approval, etc.)

Known Constraints or Assumptions ? anything explicitly out of scope, environment limitations, dependencies

Data Entities ? main objects the system stores (e.g. User, Order, Report) and key fields for each

Be as specific and exhaustive as possible ? this will be the single source of truth I use to write test scope, test strategy, and test cases, so don't leave features vague. If some sections don't apply, say so explicitly rather than skipping them.

=====

[2026-07-16T15:19:57.750794400Z] ASSISTANT

I'll inspect the repo first so the dump is grounded in the actual implementation, then I'll synthesize it under your exact headings.

=====

Project Name & One-Line Description

StockVision is an AI-powered stock market forecasting dashboard that retrieves Yahoo Finance market data, generates ARIMA and LSTM stock price forecasts, visualizes historical and predicted prices, displays technical indicators, and produces analysis reports.

Project Objectives (business goals, what problem it solves)

- Help users analyze stocks through a single visual dashboard instead of manually checking multiple finance tools.
- Provide short-term stock price forecasts using two model families: **ARIMA** and **LSTM**.
- Allow comparison between model outputs using evaluation metrics such as **MAE**, **RMSE**, and **MAPE**.
- Provide technical analysis indicators such as **RSI**, **MACD**, and **Bollinger Bands**.
- Present historical market data and future predictions in interactive charts.
- Support quick lookup of popular stocks and ETFs.
- Provide a report-style summary that turns model predictions and indicators into a simplified buy, hold, or sell recommendation.
- Allow users to export forecast data as CSV for offline analysis or report usage.
- Provide a local portfolio simulator component, although it is currently not wired into the main navigation.

Target Users / Stakeholders (roles: admin, end user, etc.)

- **End users / retail investors**
 - Use the dashboard to search tickers, view stock forecasts, compare ARIMA and LSTM predictions, inspect indicators, and generate reports.
- **Students / researchers**
 - Use the project to demonstrate software testing, AI model integration, financial data visualization, and REST API usage.
- **Financial analysts / testers**
 - Validate forecast outputs, indicators, model status, market data correctness, and report calculations.
- **Developers**
 - Maintain the React frontend, FastAPI backend, ML model loading logic, and Yahoo Finance API integration.
- **No admin role is implemented**
 - There is no login, access control, account management, or admin panel in the current project.

Functional Requirements ? list every feature/module the system has, grouped by module (e.g. Authentication, Dashboard, Data Input, Reporting, etc.), each as a short bullet

1. Application Shell / Navigation

- Display a fixed top navigation bar with the StockVision logo.
- Display the product name **StockVision**.
- Display the tagline **AI-Powered Forecasting**.
- Show application version **v1.3**.
- Provide tab navigation for:
 - **Dashboard**
 - **Markets**
 - **Reports**
- Highlight the currently active tab.
- Show the sidebar only on:

- Dashboard
- Reports
- Hide the sidebar on:
 - Markets

2. Dashboard Module

- Show a welcome screen when no forecast has been generated.
- Allow users to enter a stock ticker symbol.
- Default ticker symbol is **AAPL**.
- Default forecast horizon is **17 days**.
- Allow the user to generate a forecast manually.
- Display a loading overlay while data is being fetched.
- Display error messages when forecast generation fails.
- Allow users to dismiss forecast error messages.
- After forecast data is returned, display:
 - Stock header
 - Historical and forecast chart
 - Model evaluation metrics
 - Technical indicators

3. Sidebar / Forecast Input Module

- Provide a ticker input field.
- Convert ticker input to uppercase.
- Prevent forecast generation if the ticker is blank.
- Trigger forecast generation by:
 - Clicking the generate button
 - Pressing Enter in the input field
 - Selecting a search suggestion
- Allow forecast horizon selection from fixed options:
 - 7 days
 - 14 days
 - 17 days
 - 30 days
- Search stock suggestions through the backend `/api/search` endpoint.
- Debounce stock search requests by **200 ms**.
- Show stock suggestions when the query length is at least 1 character.
- Hide suggestions when:
 - User presses Escape
 - User clicks outside the suggestions area
 - User selects a suggestion
- Store recent searches in browser `localStorage` under key `sv_recent`.
- Limit recent searches to the latest **5** symbols.
- Check backend health/model status on sidebar mount through `/api/health`.
- Display ARIMA and LSTM model status:
 - Loading
 - Model loaded
 - Placeholder
 - Pending
 - Error / backend unreachable

4. Stock Search Module

- Backend exposes `/api/search`.
- Search is based on a curated in-memory list of popular stocks and ETFs.
- If the query is empty, return the first 10 popular stocks.

- If the query is non-empty, match against:
 - Stock symbol
 - Company name
- Search is case-insensitive.
- Return a maximum of **8** matching results for non-empty searches.
- Supported curated tickers include examples such as:
 - AAPL
 - MSFT
 - GOOGL
 - GOOG
 - AMZN
 - NVDA
 - META
 - TSLA
 - SPY
 - QQQ
 - DIA
 - IWM
 - JPM
 - V
 - NFLX
 - AMD
 - COIN
 - SHOP
 - PLTR
 - ARM

5. Forecast API Module

- Backend exposes `/api/forecast`.
- Accepts query parameters:
 - `'symbol'`, required string
 - `'days'`, optional integer, default `'17'`
- Downloads **2** years of stock data from Yahoo Finance through `'yfinance'`.
- Uses adjusted prices with `'auto_adjust=True'`.
- Returns an error object if Yahoo Finance returns no data.
- Handles MultiIndex columns from Yahoo Finance by flattening them.
- Extracts historical:
 - Close prices
 - Open prices
 - High prices
 - Low prices
 - Volume
- Generates future forecast dates using business days only, excluding Saturdays and Sundays.
- Generates ARIMA predictions.
- Generates LSTM predictions.
- Uses placeholder ARIMA predictions if the ARIMA model file is unavailable or prediction fails.
- Uses placeholder LSTM predictions if the LSTM model file is unavailable, data is insufficient, or prediction fails.
- Generates ARIMA past predictions for backtesting using placeholder logic.
- Generates LSTM past predictions using the real LSTM model if available.
- Generates placeholder LSTM past predictions if LSTM backtesting is unavailable.
- Returns technical indicators:
 - RSI
 - MACD
 - MACD signal

- Bollinger upper band
- Bollinger lower band
- Returns evaluation metrics:
 - ARIMA MAE
 - ARIMA RMSE
 - ARIMA MAPE
 - ARIMA source, either 'model' or 'placeholder'
 - LSTM MAE
 - LSTM RMSE
 - LSTM MAPE
 - LSTM source, either 'model' or 'placeholder'

6. Forecast Chart Module

- Uses Plotly via 'react-plotly.js'.
- Shows a placeholder panel when no forecast data is available.
- Displays historical stock price data as a candlestick chart.
- Displays ARIMA past prediction line.
- Displays LSTM past prediction line.
- Displays ARIMA future forecast line.
- Displays LSTM future forecast line.
- Filters out null and NaN values before plotting.
- Uses distinct styling:
 - Green candles for increasing prices
 - Red candles for decreasing prices
 - Blue ARIMA line
 - Orange LSTM line
- Shows forecast traces using markers and dashed/dotted line styles.
- Supports Plotly interactive chart behavior such as hover, zoom, and legend toggling.

7. Stock Header Module

- Displays the selected stock symbol.
- Displays company/exchange labels for known symbols:
 - AAPL
 - TSLA
 - GOOGL
 - MSFT
 - AMZN
 - META
 - NVDA
 - NFLX
 - BTC
- Displays fallback label as '{SYMBOL} • STOCK'.
- Displays latest close price.
- Calculates and displays one-day price change.
- Calculates and displays one-day percentage change.
- Styles positive changes in green.
- Styles negative changes in red.
- Provides time range tabs:
 - 1W
 - 1M
 - 3M
 - 6M
 - 1Y
 - 2Y
- Tracks selected time range in component state.

- Provides CSV export functionality.
- CSV export includes:
 - Date
 - Open
 - High
 - Low
 - Close
 - Volume
 - Type
 - ARIMA Forecast
 - LSTM Forecast
- CSV contains both historical rows and forecast rows.
- CSV file name format is:
 - '{SYMBOL}_forecast_{YYYY-MM-DD}.csv'

8. Model Evaluation Module

- Shows a placeholder message before forecast generation.
- Displays ARIMA evaluation metrics.
- Displays LSTM evaluation metrics.
- Displays:
 - MAE
 - RMSE
 - MAPE
- Shows the forecast horizon in days.
- Compares ARIMA and LSTM using MAE.
- Marks the model with the lower MAE as the better model.
- Displays a verdict indicating whether ARIMA or LSTM performs better.
- Uses green highlighting for the selected better model.

9. Technical Indicators Module

- Shows a placeholder message before forecast generation.
- Allows users to toggle indicators:
 - RSI
 - MACD
 - Bollinger Bands
- Default active indicator is **RSI**.
- RSI chart:
 - Displays RSI line.
 - Uses 0 to 100 y-axis range.
 - Shows overbought threshold at 70.
 - Shows oversold threshold at 30.
 - Labels overbought and oversold zones.
- MACD chart:
 - Displays MACD line.
 - Displays MACD signal line.
 - Displays histogram difference between MACD and signal.
 - Uses green bars for positive histogram values.
 - Uses red bars for negative histogram values.
- Bollinger Bands chart:
 - Displays close price.
 - Displays upper Bollinger band.
 - Displays lower Bollinger band.
- Filters invalid null or NaN values before plotting.

10. Markets Overview Module

- Backend exposes '/api/markets'.
- Frontend displays a Markets Overview page.
- Automatically loads market data on page mount.
- Shows loading state while market data is fetched.
- Shows an error state if market data fails.
- Fetches market overview for:
 - SPY
 - QQQ
 - DIA
 - AAPL
 - TSLA
 - NVDA
 - MSFT
 - GOOGL
 - AMZN
- Downloads 5 days of Yahoo Finance data.
- Displays table columns:
 - Ticker
 - Name
 - Price
 - Change
 - % Change
 - High
 - Low
 - Action
- Calculates current price from latest close.
- Calculates price change from previous close.
- Calculates percentage change.
- Displays positive change in green.
- Displays negative change in red.
- Allows the user to click a market row to generate a forecast for that symbol.
- Includes Forecast action behavior through 'onForecastSelect'.

11. Reports Module

- Shows a ****No Forecast Loaded**** message when no forecast data exists.
- Requires a forecast to be generated before report content is shown.
- Uses the current forecast data from the Dashboard.
- Calculates latest close price.
- Calculates ARIMA target as the final ARIMA forecast value.
- Calculates ARIMA percentage change against latest close.
- Classifies ARIMA signal:
 - Bullish if change > 1.0%
 - Bearish if change < -1.0%
 - Neutral otherwise
- Calculates LSTM target as the final LSTM forecast value.
- Calculates LSTM percentage change against latest close.
- Classifies LSTM signal:
 - Bullish if change > 1.0%
 - Bearish if change < -1.0%
 - Neutral otherwise
- Reads latest RSI value.
- Classifies RSI:
 - Overbought if RSI >= 70
 - Oversold if RSI <= 30
 - Neutral otherwise

- Reads latest MACD and MACD signal values.
- Calculates MACD difference.
- Classifies MACD:
 - Bullish if MACD difference > 0.05
 - Bearish if MACD difference < -0.05
 - Neutral otherwise
- Reads latest Bollinger upper and lower bands.
- Classifies Bollinger Bands:
 - Overbought if latest close >= upper band × 0.97
 - Oversold if latest close <= lower band × 1.03
 - Neutral otherwise
- Calculates an overall score:
 - ARIMA Bullish: +2
 - ARIMA Bearish: -2
 - LSTM Bullish: +2
 - LSTM Bearish: -2
 - RSI Oversold: +1
 - RSI Overbought: -1
 - MACD Bullish: +1
 - MACD Bearish: -1
 - Bollinger Oversold: +1
 - Bollinger Overbought: -1
- Converts score into recommendation:
 - Score >= 4: Strong Buy
 - Score >= 1: Buy
 - Score <= -4: Strong Sell
 - Score <= -1: Sell
 - Otherwise: Hold
- Provides a printable report using 'window.print()'.
 - Defines print-specific CSS to hide navigation, sidebar, buttons, and non-print UI.

12. Portfolio Simulator Module

- A 'PortfolioView.jsx' component exists, but it is **not** currently connected to the main navigation in 'App.jsx'.
- If integrated, it supports local portfolio simulation.
- Starts with default cash balance of **\$100,000.00**.
- Stores portfolio state in browser 'localStorage'.
- Uses localStorage keys:
 - 'stockvision_cash'
 - 'stockvision_holdings'
 - 'stockvision_transactions'
- Supports buy transactions.
- Supports sell transactions.
- Tracks holdings.
- Tracks transaction history.
- Fetches live prices for held symbols using '/api/prices'.
- Refreshes live prices every **30 seconds**.
- Calculates portfolio value.
- Calculates market value.
- Calculates gain/loss.
- Allows portfolio reset after confirmation.
- Validates transaction inputs:
 - Symbol is required.
 - Shares must be a positive integer.
 - Price must be a positive number.
 - Buy requires enough cash.

- Sell requires enough owned shares.

13. Backend Health Module

- Backend exposes '/api/health'.
- Returns general status:
 - 'status: "ok"'
- Returns ARIMA model health:
 - Status
 - Loaded boolean
 - Model file path
 - Message
- Returns LSTM model health:
 - Status
 - Loaded boolean
 - Model file path
 - Message
- Reports model as 'ready' if the model file exists.
- Reports model as 'placeholder' if the model file does not exist.

14. Prices API Module

- Backend exposes '/api/prices'.
- Accepts query parameter:
 - 'symbols', required comma-separated string
- Normalizes symbols by trimming whitespace and converting to uppercase.
- Returns an empty object if no symbols are provided after parsing.
- Downloads 5 days of data from Yahoo Finance.
- Returns the latest close price for each requested symbol.
- Handles single-symbol and multi-symbol Yahoo Finance response formats.
- Returns '{error: message}' if an exception occurs.

15. Authentication Module

- Not implemented.
- No login page exists.
- No registration page exists.
- No password reset exists.
- No sessions or JWT tokens exist.
- No backend user model exists.
- No role-based access control exists.

16. Database / Persistence Module

- No server-side database is implemented.
- Forecasts are not stored permanently.
- Users are not stored permanently.
- Reports are not stored permanently.
- Portfolio data, if the unused PortfolioView is integrated, is stored client-side in 'localStorage'.
- Recent searches are stored client-side in 'localStorage'.

Non-Functional Requirements ? performance, security, usability, scalability, availability targets if any

Performance

- Frontend uses **React 18** with **Vite 5**, which supports fast development builds and optimized production builds.
- Backend uses **FastAPI**, which is suitable for lightweight REST APIs.
- Stock search suggestions are debounced by **200 ms** to reduce excessive API calls.
- LSTM model is loaded lazily and cached in memory after first load.
- ARIMA model is loaded lazily and cached in memory after first load.
- Portfolio live prices refresh every **30 seconds**.
- Market overview requests fetch only 5 days of data.
- Forecast endpoint fetches 2 years of data, which may be slower depending on Yahoo Finance availability and network speed.
- No explicit response time target is documented.

Security

- No authentication or authorization exists.
- CORS is restricted to local frontend origins:
 - 'http://localhost:5173'
 - 'http://localhost:5174'
 - 'http://localhost:4173'
- API accepts public ticker input and calls Yahoo Finance.
- No secret keys or credentials are required.
- No server-side user data is stored.
- No rate limiting is implemented.
- No input sanitization beyond basic trimming/uppercasing is implemented for stock symbols.
- No HTTPS configuration is included in the project.
- No CSRF protection is needed for the current stateless public GET endpoints, but it is also not implemented.

Usability

- Simple tab-based navigation.
- Loading states are provided for forecasts and market data.
- Error banners are provided for forecast failures.
- Search suggestions assist ticker discovery.
- Recent searches improve repeated access.
- Chart placeholders explain what users should do before data exists.
- Model status indicators tell users whether predictions use actual models or placeholders.
- CSV export supports offline use.
- Printable reports support documentation and presentation use.

Reliability / Availability

- Backend continues running even when model files are missing.
- Placeholder predictions are used when ARIMA or LSTM models are unavailable.
- Forecast endpoint returns error objects instead of crashing for many failure cases.
- Heavy model dependencies are imported lazily.
- External dependency on Yahoo Finance may cause failures or inconsistent data availability.
- No retry logic is implemented for Yahoo Finance failures.
- No uptime or availability target is documented.

Scalability

- Current implementation is intended for local/development use.
- No database scaling concern exists because no database is used.
- ML model inference runs inside the FastAPI process.
- Yahoo Finance calls happen synchronously during API requests.
- No job queue, caching layer, worker process, or distributed inference system is

implemented.

- Concurrent heavy forecast requests may affect backend responsiveness.
- No production deployment architecture is included.

Maintainability

- Frontend is componentized by feature:
 - Navbar
 - Sidebar
 - ForecastChart
 - StockHeader
 - EvaluationMetrics
 - TechnicalIndicators
 - MarketsView
 - ReportsView
 - PortfolioView
- Backend is currently implemented in a single 'main.py' file.
- Backend may become harder to maintain as APIs, model logic, and indicator calculations grow.
- No automated tests are present in the repository.
- No linting or formatting configuration is present.

Compatibility

- Backend requires **Python 3.9+.**
- Frontend uses modern React and Vite tooling.
- Intended local frontend URL is 'http://localhost:5173'.
- Intended local backend URL is 'http://localhost:8000'.
- Frontend API base URL can be overridden using 'VITE_API_URL'.

Business Rules / Validation Rules ? e.g. field constraints, workflow rules, access control rules

Forecast Input Rules

- Stock symbol is required for forecast generation.
- Blank or whitespace-only ticker input does not trigger forecast fetching.
- Ticker input is trimmed before API call.
- Ticker input is converted to uppercase in the UI.
- Forecast days should be selected from:
 - 7
 - 14
 - 17
 - 30
- Backend 'days' parameter defaults to 17 if omitted.
- Backend does not enforce a strict maximum for 'days', which is a test risk.
- Backend does not explicitly reject zero or negative 'days', which is a test risk.
- If Yahoo Finance returns no data for a symbol, backend returns:
 - '{"error": "No data found for symbol '{symbol}'"}'

Forecast Date Rules

- Future forecast dates are generated as business days only.
- Saturday and Sunday are skipped.
- Market holidays are not explicitly skipped.

Model Rules

- ARIMA model expected path:
 - 'backend/models/arma_universal.pkl'
- LSTM model expected path in code:
 - 'backend/models/lstm_stock_model.h5'
- Some documentation also mentions:
 - 'backend/models/lstm_universal.h5'
- If ARIMA model is missing, fails loading, or fails predicting, use placeholder ARIMA predictions.
- If LSTM model is missing, fails loading, has insufficient data, or fails predicting, use placeholder LSTM predictions.
- LSTM requires at least **60** historical close values for its prediction window.
- LSTM backtest uses a sliding 60-day window.
- Backtest display is limited to **90 days** where enough data exists.
- Evaluation metrics are currently generated using random seeded placeholder ranges, not true model validation metrics.

Technical Indicator Rules

- RSI uses a default period of **14**.
- RSI missing values are filled with **50**.
- RSI overbought threshold is **70**.
- RSI oversold threshold is **30**.
- MACD uses:
 - Fast EMA period: 12
 - Slow EMA period: 26
 - Signal EMA period: 9
- Bollinger Bands use:
 - Period: 20
 - Standard deviation multiplier: 2.0
- Bollinger missing initial values are backfilled.

Report Recommendation Rules

- ARIMA signal:
 - Bullish if final ARIMA forecast is more than 1% above latest close.
 - Bearish if final ARIMA forecast is more than 1% below latest close.
 - Neutral otherwise.
- LSTM signal:
 - Bullish if final LSTM forecast is more than 1% above latest close.
 - Bearish if final LSTM forecast is more than 1% below latest close.
 - Neutral otherwise.
- RSI signal:
 - Overbought if RSI ≥ 70 .
 - Oversold if RSI ≤ 30 .
 - Neutral otherwise.
- MACD signal:
 - Bullish if MACD minus signal > 0.05 .
 - Bearish if MACD minus signal < -0.05 .
 - Neutral otherwise.
- Bollinger signal:
 - Overbought if latest close $\geq$ upper band $\times 0.97$.
 - Oversold if latest close $\leq$ lower band $\times 1.03$.
 - Neutral otherwise.
- Overall report score:
 - ARIMA Bullish: +2
 - ARIMA Bearish: -2

- LSTM Bullish: +2
- LSTM Bearish: -2
- RSI Oversold: +1
- RSI Overbought: -1
- MACD Bullish: +1
- MACD Bearish: -1
- Bollinger Oversold: +1
- Bollinger Overbought: -1
- Final recommendation:
 - Score $\geq$ 4: Strong Buy
 - Score $\geq$ 1: Buy
 - Score $\leq$ -4: Strong Sell
 - Score $\leq$ -1: Sell
 - Otherwise: Hold

Market Overview Rules

- Market overview fixed symbols:
 - SPY
 - QQQ
 - DIA
 - AAPL
 - TSLA
 - NVDA
 - MSFT
 - GOOGL
 - AMZN
- Market overview uses the latest and previous close values to calculate change and percentage change.
- Symbols with insufficient data are skipped.
- If Yahoo Finance market data is empty, backend returns:
 - '{"error": "Failed to fetch market data"}'

Search Rules

- Empty search query returns first 10 curated popular stocks.
- Non-empty search query returns up to 8 matches.
- Search matches against symbol or company name.
- Search is case-insensitive.

Portfolio Simulator Rules

- Applies only if the unused PortfolioView is integrated.
- Starting cash is **\$100,000.00**.
- Symbol is required.
- Shares must be a positive integer.
- Price must be a positive number.
- Buy transaction cannot exceed available cash.
- Sell transaction cannot exceed owned shares.
- Buying more of an existing holding recalculates average buy price.
- Selling all shares removes the holding.
- Portfolio reset asks for confirmation.
- Portfolio data is persisted in localStorage only.

Access Control Rules

- No access control rules are implemented.

- All frontend views and backend endpoints are publicly accessible in the local app.

System Architecture Summary ? frontend stack, backend stack, database, APIs, third-party integrations

Frontend

- Framework:
 - React 18
- Build tool:
 - Vite 5
- Language:
 - JavaScript / JSX
- Charting:
 - Plotly.js
 - react-plotly.js
- HTTP client:
 - Native 'fetch'
 - Axios is installed but not used in 'api.js'
- Styling:
 - Inline React styles
 - Shared color constants from 'frontend/src/colors.js'
- Main entry points:
 - 'frontend/src/main.jsx'
 - 'frontend/src/App.jsx'
- Components:
 - 'Navbar.jsx'
 - 'Sidebar.jsx'
 - 'StockHeader.jsx'
 - 'ForecastChart.jsx'
 - 'EvaluationMetrics.jsx'
 - 'TechnicalIndicators.jsx'
 - 'MarketsView.jsx'
 - 'ReportsView.jsx'
 - 'PortfolioView.jsx'
- Frontend API service:
 - 'frontend/src/services/api.js'
- Frontend default URL:
 - 'http://localhost:5173'
- Environment variable:
 - 'VITE_API_URL'
- Default API base if environment variable is missing:
 - 'http://localhost:8000'

Backend

- Framework:
 - FastAPI
- Language:
 - Python 3.9+
- Server:
 - Uvicorn
- Main file:
 - 'backend/main.py'
- Backend default URL:
 - 'http://localhost:8000'
- Interactive API docs:

- 'http://localhost:8000/docs'
- CORS:
 - Enabled through FastAPI 'CORSMiddleware'
 - Allowed local origins:
 - 'http://localhost:5173'
 - 'http://localhost:5174'
 - 'http://localhost:4173'

Backend Libraries

- 'fastapi'
- 'uvicorn[standard]'
- 'yfinance'
- 'pandas'
- 'numpy'
- 'python-multipart'
- 'statsmodels'
- 'scikit-learn'
- 'tensorflow'

Machine Learning / Forecasting

- ARIMA:
 - Loaded from pickle file.
 - Expected code path:
 - 'backend/models/arima_universal.pkl'
 - Supports model objects exposing:
 - 'forecast(steps=days)'
 - 'predict(n_periods=days)'
 - 'predict(days)'
- LSTM:
 - Loaded using TensorFlow Keras.
 - Expected code path:
 - 'backend/models/lstm_stock_model.h5'
 - Uses MinMaxScaler.
 - Uses last 60 days as input window.
 - Performs iterative multi-step prediction.
- Placeholder models:
 - Used when actual model files are missing or fail.
 - Placeholder ARIMA uses seeded random walk with seed 42.
 - Placeholder LSTM uses seeded random walk with seed 7.

Database

- No database is implemented.
- No SQL, NoSQL, ORM, or migration tool exists.
- Persistent frontend-only storage:
 - Recent searches in localStorage.
 - Portfolio simulator state in localStorage if integrated.

APIs

- 'GET /api/health'
 - Returns API status and model readiness.
- 'GET /api/search?q=...'
- Searches curated stock list.
- 'GET /api/markets'

- Returns market overview for fixed list of symbols.
- 'GET /api/prices?symbols=AAPL,MSFT'
- Returns latest prices for requested tickers.
- 'GET /api/forecast?symbol=AAPL&days=17'
- Returns historical data, forecasts, indicators, backtest data, and evaluation metrics.

Third-Party Integrations

- Yahoo Finance through the 'yfinance' Python package.
- TensorFlow/Keras for LSTM model loading.
- scikit-learn for MinMaxScaler.
- Plotly for interactive frontend charting.

Deployment / Runtime

- Local development requires two terminals:
 - Backend:
 - 'cd backend'
 - 'pip install -r requirements.txt'
 - 'uvicorn main:app --reload'
 - Frontend:
 - 'cd frontend'
 - 'npm install'
 - 'npm run dev'
- No Dockerfile exists.
- No CI/CD pipeline exists.
- No production deployment config exists.

User Roles & Permissions ? what each role can/cannot do

Anonymous End User

Can:

- Open the application.
- Navigate between Dashboard, Markets, and Reports.
- Search for stock tickers.
- Select forecast horizon.
- Generate stock forecasts.
- View candlestick charts.
- View ARIMA and LSTM predictions.
- View technical indicators.
- View model evaluation metrics.
- View market overview.
- Click a market symbol to generate a forecast.
- Export forecast data to CSV.
- Print reports.
- Use browser-local recent search history.
- Use portfolio simulator features only if PortfolioView is connected to the app.

Cannot:

- Log in, because authentication is not implemented.
- Create an account.
- Save server-side forecasts.
- Save server-side reports.
- Manage users.

- Configure model files from the UI.
- Upload model files.
- Modify backend stock lists from the UI.
- Access admin features, because none exist.

Admin

- Not applicable.
- No admin role exists.
- No admin-specific permissions exist.

Developer / Maintainer

Can be by modifying source code or files:

- Start backend and frontend locally.
- Add or replace model files in 'backend/models'.
- Change curated stock list in 'backend/main.py'.
- Modify CORS settings.
- Modify technical indicator calculations.
- Add tests, deployment, database, or authentication.

Key User Workflows / Use Cases ? step-by-step for the top 5-8 core flows (e.g. login, submit form, generate report, admin approval, etc.)

1. Generate a Stock Forecast from Dashboard

1. User opens the frontend at 'http://localhost:5173'.
2. App loads the Dashboard tab by default.
3. Sidebar appears with default symbol 'AAPL' and default forecast horizon '17'.
4. Sidebar checks backend model health using '/api/health'.
5. User enters a ticker symbol, for example 'TSLA'.
6. User selects forecast horizon:
 - 7
 - 14
 - 17
 - 30
7. User clicks generate or presses Enter.
8. Frontend calls '/api/forecast?symbol=TSLA&days={selectedDays}'.
9. Backend downloads 2 years of Yahoo Finance data.
10. Backend generates business-day forecast dates.
11. Backend generates ARIMA predictions or placeholder ARIMA predictions.
12. Backend generates LSTM predictions or placeholder LSTM predictions.
13. Backend calculates technical indicators.
14. Backend returns historical prices, OHLCV, forecasts, indicators, and metrics.
15. Frontend stores result in state.
16. Dashboard displays stock header, forecast chart, evaluation metrics, and technical indicators.

2. Search and Select a Stock Suggestion

1. User focuses ticker input in the sidebar.
2. User types at least one character.
3. Input is converted to uppercase.
4. Frontend waits 200 ms after input change.
5. Frontend calls '/api/search?q={query}'.
6. Backend searches curated popular stock list.

7. Backend returns up to 8 matching stocks.
8. Frontend displays suggestions.
9. User clicks a suggestion.
10. Frontend sets selected symbol.
11. Symbol is saved to recent searches in localStorage.
12. Suggestions are hidden.
13. Forecast is automatically generated for the selected symbol.

3. View Market Overview and Forecast a Market Symbol

1. User clicks the Markets tab.
2. Sidebar is hidden.
3. MarketsView loads and calls '/api/markets'.
4. Backend downloads 5 days of Yahoo Finance data for fixed symbols.
5. Backend calculates current price, change, percentage change, high, and low.
6. Frontend displays the market overview table.
7. User clicks a market row or forecast action.
8. App switches back to Dashboard.
9. App sets the selected symbol.
10. App generates a forecast for that symbol.
11. Dashboard displays forecast results.

4. Analyze Model Evaluation Metrics

1. User generates a forecast.
2. Backend returns ARIMA and LSTM evaluation objects.
3. EvaluationMetrics component reads:
 - MAE
 - RMSE
 - MAPE
 - Source
4. Component compares ARIMA MAE against LSTM MAE.
5. Model with lower MAE is marked as the better model.
6. User sees ARIMA and LSTM metric cards.
7. User sees a verdict indicating which model performs better.

5. Toggle Technical Indicators

1. User generates a forecast.
2. TechnicalIndicators component receives indicator data.
3. Default active indicator is RSI.
4. User toggles RSI, MACD, or Bollinger Bands.
5. Component updates selected indicator state.
6. Relevant Plotly chart traces are shown or hidden.
7. User interprets overbought, oversold, bullish, bearish, or neutral signals.

6. Export Forecast Data to CSV

1. User generates a forecast.
2. StockHeader receives forecast data.
3. User clicks export.
4. Frontend builds CSV rows for historical data.
5. Frontend appends forecast rows.
6. Frontend creates a CSV Blob.
7. Browser downloads the file named:
 - '{SYMBOL}_forecast_{YYYY-MM-DD}.csv'
8. User can use the CSV in Excel, reports, or testing documentation.

7. Generate and Print a Report

1. User first generates a forecast on the Dashboard.
2. User clicks the Reports tab.
3. ReportsView receives current forecast data.
4. Component calculates ARIMA, LSTM, RSI, MACD, and Bollinger signals.
5. Component calculates total recommendation score.
6. Component converts score into:
 - Strong Buy
 - Buy
 - Hold
 - Sell
 - Strong Sell
7. User reviews report summary.
8. User clicks print.
9. Browser print dialog opens through 'window.print()'.
10. Print CSS hides navigation/sidebar/buttons and formats report output.

8. Portfolio Simulation Workflow, if Integrated

1. User opens PortfolioView, although it is currently not linked in the main navigation.
2. Component loads cash, holdings, and transactions from localStorage.
3. User enters symbol, shares, price, and transaction type.
4. User submits buy or sell.
5. Component validates symbol, shares, price, cash, and holdings.
6. Component updates cash and holdings.
7. Component records transaction history.
8. Component saves updated state to localStorage.
9. Component fetches live prices for holdings every 30 seconds.
10. User can reset portfolio after confirmation.

Known Constraints or Assumptions ? anything explicitly out of scope, environment limitations, dependencies

- The system is designed primarily for local development.
- Backend must run separately from frontend.
- Backend default URL is 'http://localhost:8000'.
- Frontend default URL is 'http://localhost:5173'.
- Frontend uses 'VITE_API_URL' if provided.
- Yahoo Finance availability is required for real market and forecast data.
- The app depends on external internet access for 'yfinance'.
- There is no database.
- There is no authentication.
- There is no admin role.
- There is no server-side persistence of forecasts, reports, or users.
- Portfolio simulator is present in code but not wired into 'App.jsx' navigation.
- Model files may be missing. The app handles this with placeholders.
- Code expects LSTM file at 'backend/models/lstm_stock_model.h5'.
- Backend README mentions 'lstm_universal.h5', which conflicts with the code.
- Root README mentions 'backend/models/arima_universal.pkl' and 'backend/models/lstm_stock_model.h5'.
- Current repository contains cached ARIMA files such as 'arima_AAPL.pkl', 'arima_AMZN.pkl', etc., but the backend code currently looks for 'arima_universal.pkl'.
- Forecast evaluation metrics are not true measured model performance. They are generated from seeded random ranges.
- ARIMA past predictions are placeholder-based.

- Market holidays are not considered when generating forecast dates.
- Weekends are skipped.
- There are no automated tests in the project.
- There is no testing framework configured for frontend or backend.
- There is no Docker setup.
- There is no CI/CD setup.
- There is no production deployment configuration.
- There is no explicit non-functional performance target.
- There is no rate limiting.
- Backend endpoints mostly return '{error: ...}' with HTTP 200 instead of using error HTTP status codes.
- Financial predictions are for demonstration and should not be treated as professional investment advice.
- The system does not guarantee model accuracy.
- The curated stock search list is hardcoded and not dynamically synced with an exchange listing.
- Time range buttons in StockHeader update UI state but do not appear to filter the displayed chart data.
- Axios is installed but the API service uses native 'fetch'.

Data Entities ? main objects the system stores (e.g. User, Order, Report) and key fields for each

User

- Not implemented.
- No user entity exists.
- No user fields exist.

Stock Search Suggestion

Stored in backend memory as part of 'POPULAR_STOCKS'.

Key fields:

- 'symbol'
 - Example: 'AAPL'
- 'name'
 - Example: 'Apple Inc.'

Recent Search

Stored in browser 'localStorage' under 'sv_recent'.

Key fields:

- 'symbol'
 - Stock ticker searched or selected.
- 'timestamp'
 - JavaScript timestamp from 'Date.now()'.

Forecast Request

Not stored persistently. Sent as API query parameters.

Key fields:

- 'symbol'
 - Required stock ticker.
- 'days'
 - Number of forecast business days.
 - Default: 17.

Forecast Response

Returned by '/api/forecast'.

Key fields:

- 'symbol'
 - Uppercase ticker symbol.
- 'dates'
 - Historical date list.
- 'historical_prices'
 - Historical close prices.
- 'opens'
 - Historical open prices.
- 'highs'
 - Historical high prices.
- 'lows'
 - Historical low prices.
- 'volumes'
 - Historical volume values.
- 'forecast_dates'
 - Future business-day dates.
- 'arma_predictions'
 - Future ARIMA forecast values.
- 'lstm_predictions'
 - Future LSTM forecast values.
- 'arma_past_predictions'
 - Past ARIMA prediction/backtest values.
- 'arma_past_dates'
 - Dates for ARIMA past predictions.
- 'lstm_past_predictions'
 - Past LSTM prediction/backtest values.
- 'lstm_past_dates'
 - Dates for LSTM past predictions.
- 'evaluation'
 - Model evaluation object.
- 'indicators'
 - Technical indicators object.

Evaluation

Nested inside forecast response.

Key fields:

- 'arma'
 - 'mae'
 - 'rmse'
 - 'mape'
 - 'source'
- 'lstm'

- 'mae'
- 'rmse'
- 'mape'
- 'source'

Technical Indicators

Nested inside forecast response.

Key fields:

- 'rsi'
- 'macd'
- 'macd_signal'
- 'bollinger_upper'
- 'bollinger_lower'

Market Overview Item

Returned by '/api/markets'.

Key fields:

- 'symbol'
- 'name'
- 'price'
- 'change'
- 'change_percent'
- 'high'
- 'low'

Price Lookup Response

Returned by '/api/prices'.

Structure:

- Object/dictionary where each key is a ticker symbol.
- Each value is the latest close price.

Example fields:

- 'AAPL: 123.45'
- 'MSFT: 456.78'

Report

Not stored persistently. Computed in the frontend from forecast data.

Derived fields:

- 'symbol'
- 'latestClose'
- 'arimaTarget'
- 'arimaChange'
- 'arimaSignal'
- 'IstmTarget'

- 'IstmChange'
- 'IstmSignal'
- 'latestRsi'
- 'rsiSignal'
- 'latestMacd'
- 'latestSignal'
- 'macdDiff'
- 'macdSignal'
- 'upperB'
- 'lowerB'
- 'bbSignal'
- 'score'
- 'rating'
- 'ratingColor'
- 'ratingText'
- 'needleAngle'

Portfolio Holding

Stored in browser 'localStorage' under 'stockvision_holdings' if PortfolioView is used.

Key fields:

- 'symbol'
- 'shares'
- 'buyPrice'

Portfolio Transaction

Stored in browser 'localStorage' under 'stockvision_transactions' if PortfolioView is used.

Key fields:

- 'id'
- 'symbol'
- 'shares'
- 'price'
- 'type'
 - 'BUY'
 - 'SELL'
- 'date'

Portfolio Cash

Stored in browser 'localStorage' under 'stockvision_cash' if PortfolioView is used.

Key field:

- Cash balance as stringified number.

Model Status

Returned by '/api/health'.

Key fields:

- 'status'

- Overall API status.
- 'models.arima.status'
 - 'ready' or 'placeholder'.
- 'models.arima.loaded'
 - Boolean.
- 'models.arima.path'
 - Model file path.
- 'models.arima.message'
 - Human-readable status.
- 'models.lstm.status'
 - 'ready' or 'placeholder'.
- 'models.lstm.loaded'
 - Boolean.
- 'models.lstm.path'
 - Model file path.
- 'models.lstm.message'
 - Human-readable status.

=====

[2026-07-16T15:28:03.545379300Z] USER

You have 1 incomplete todo. Continue working, or update the todo tool.

=====

[2026-07-16T15:28:22.927403300Z] ASSISTANT

Todo updated. All project knowledge dump tasks are now marked completed.

=====

[2026-07-16T15:39:51.550964100Z] USER

? End users / retail investors

=====

[2026-07-16T15:40:00.614980800Z] ASSISTANT

****End users / retail investors**** are the primary users of StockVision. They use the system to search stock tickers, generate ARIMA/LSTM forecasts, view market trends and technical indicators, compare prediction models, export forecast data, and read simplified buy/hold/sell-style analysis reports.

=====

[2026-07-16T15:40:00.616610300Z] USER

? Use the dashboard to search tickers, view stock forecasts, compare ARIMA and LSTM predictions, inspect indicators, and generate reports.

? Students / researchers

? Use the project to demonstrate software testing, AI model integration, financial data visualization, and REST API usage.

? Financial analysts / testers

? Validate forecast outputs, indicators, model status, market data correctness, and report calculations.

? Developers

? Maintain the React frontend, FastAPI backend, ML model loading logic, and Yahoo Finance API integration.

? No admin role is implemented

? There is no login, access control, account management, or admin panel in the current project.

Functional Requirements ? list every feature/module the system has, grouped by module (e.g. Authentication, Dashboard, Data Input, Reporting, etc.), each as a short bullet

1. Application Shell / Navigation

? Display a fixed top navigation bar with the StockVision logo.

? Display the product name StockVision.

? Display the tagline AI-Powered Forecasting.

? Show application version v1.3.

? Provide tab navigation for:

? Dashboard

? Markets

? Reports

? Highlight the currently active tab.

? Show the sidebar only on:

? Dashboard

? Reports

? Hide the sidebar on:

? Markets

2. Dashboard Module

? Show a welcome screen when no forecast has been generated.

? Allow users to enter a stock ticker symbol.

? Default ticker symbol is AAPL.

? Default forecast horizon is 17 days.

? Allow the user to generate a forecast manually.

? Display a loading overlay while data is being fetched.

? Display error messages when forecast generation fails.

? Allow users to dismiss forecast error messages.

? After forecast data is returned, display:

? Stock header

? Historical and forecast chart

? Model evaluation metrics

? Technical indicators

3. Sidebar / Forecast Input Module

? Provide a ticker input field.

? Convert ticker input to uppercase.